

#放一些比较重要的代码和 plot 结果

```
# Load the netCDF4 file
```

```
ds = xr.open_dataset("200301_202006-C3S-L3_GHG-PRODUCTS-OBS4MIPS-MERGED-v4.3.nc", engine="netcdf4")
```

```
# Check the data
```

```
Ds
```

```
# Remove missing data
```

```
xch41 = ds.xch4.where((ds['xch4'] != 1.000000e+20))
```

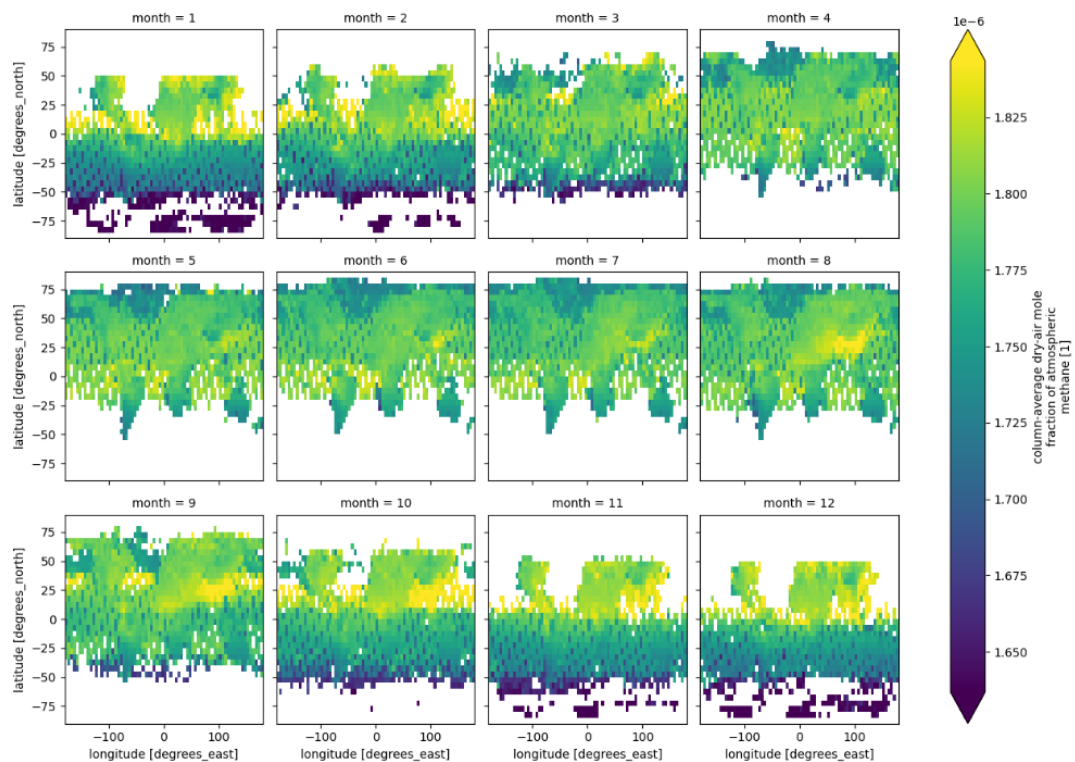
```
#1.1
```

```
xch4_clim = xch41.groupby('time.month')
```

```
# Plot monthly mean, one at a panel
```

```
xch4_clim.mean(dim='time').plot(col="month", col_wrap=4, robust=True)
```

```
plt.show()
```



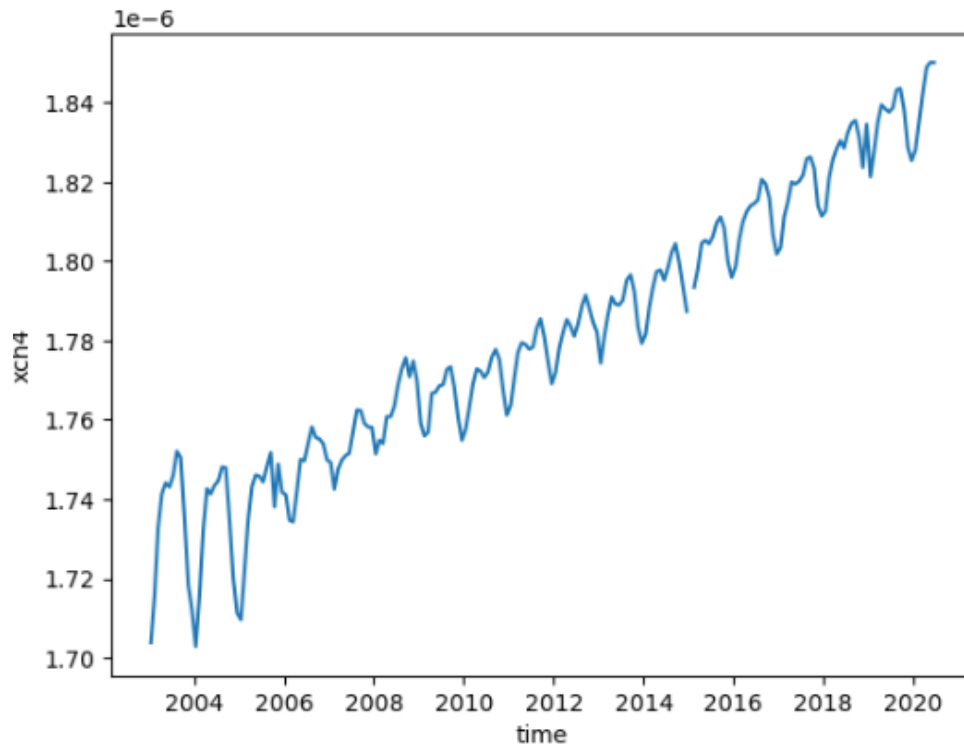
```
#1.2
```

```
ds2 = xch41.mean(dim=['lat','lon'])
```

```
ds2.plot(size=5)
```

```
plt.show()
```

```
#与 check plot 相近
```



线性插值缺失值

#对于大块缺失部分的插值太不贴合已有趋势了，如果不是用 cubic，是用 linear 线性插值呢？

#感觉这里用线性插值得好只是因为缺失部分的线性趋势刚好与整个趋势相近。？。。

2. 提取时间索引和数值（转为 numpy 数组，手动处理插值+外推）

time_coords = ds_1.time.values # 时间坐标 (datetime64 类型)

values = ds_1.values # 数据值 (1D 数组)

3. 实现“插值+外推”

def interpolate_with_extrapolate(time_coords, values, kind='linear'):

步骤 1: 提取非缺失值的索引和数值

non_nan_idx = np.where(~np.isnan(values))[0]

non_nan_vals = values[non_nan_idx]

non_nan_times = time_coords[non_nan_idx]

步骤 2: 生成插值函数（支持外推）

用时间坐标的整数索引做插值（避免 datetime 类型兼容问题）

f = interp1d(

non_nan_idx, # 非缺失值的位置索引 (0,1,2,...)

non_nan_vals,

kind=kind,

fill_value="extrapolate", # scipy 支持直接外推

bounds_error=False # 超出范围时不报错，使用外推值

)

步骤 3: 对所有时间点插值 (包括首尾缺失值)

all_idx = np.arange(len(time_coords)) # 所有数据的位置索引

filled_vals = f(all_idx)

步骤 4: 转为 DataArray (保持原始结构)

```
filled_da = xr.DataArray(  
    filled_vals,  
    coords={'time': time_coords},  
    dims=['time'],  
    name=ds_1.name
```

)

return filled_da

4. 执行插值+外推 (根据有效数据量选择方法)

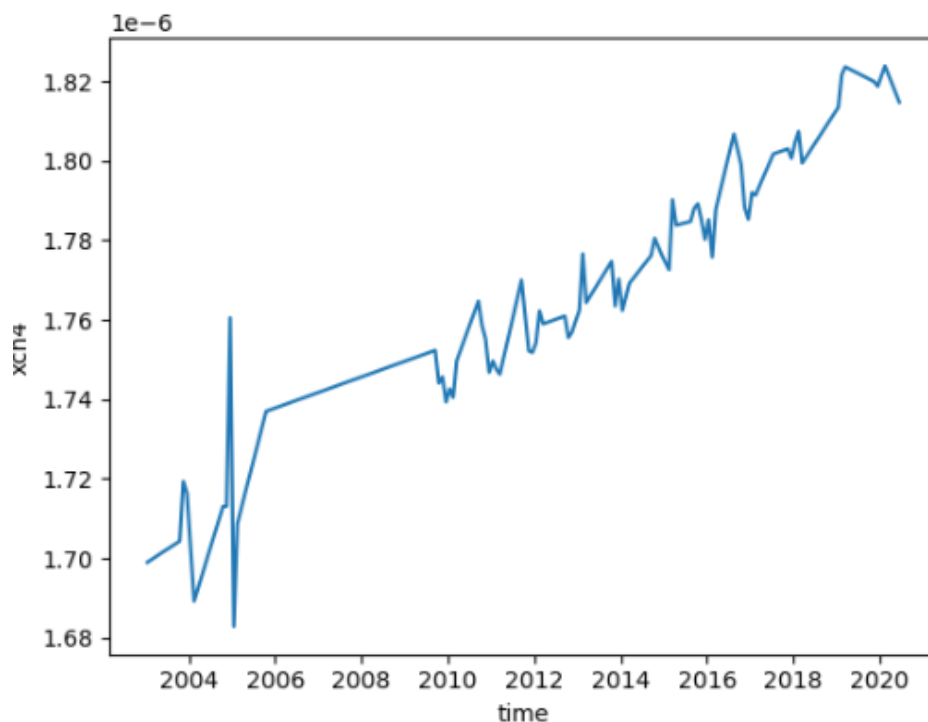
valid_count = np.count_nonzero(~np.isnan(ds_1.values))

有效数据 ≥ 3 个: 用三次插值 (更平滑)

filled = interpolate_with_extrapolate(time_coords, values, kind='linear')

filled.plot()

plt.show()



#后续季节性分解 (与之前一致)

stl = STL(filled, seasonal = 13, period = 12)

result1 = stl.fit()

result1.plot()

plt.show()

分解结果

```
trend = result1.trend
```

```
seasonal = result1.seasonal
```

```
residual = result1.resid
```

4. 去除季节性（原始数据 - 季节性成分）

```
deseasonalized = filled - seasonal
```

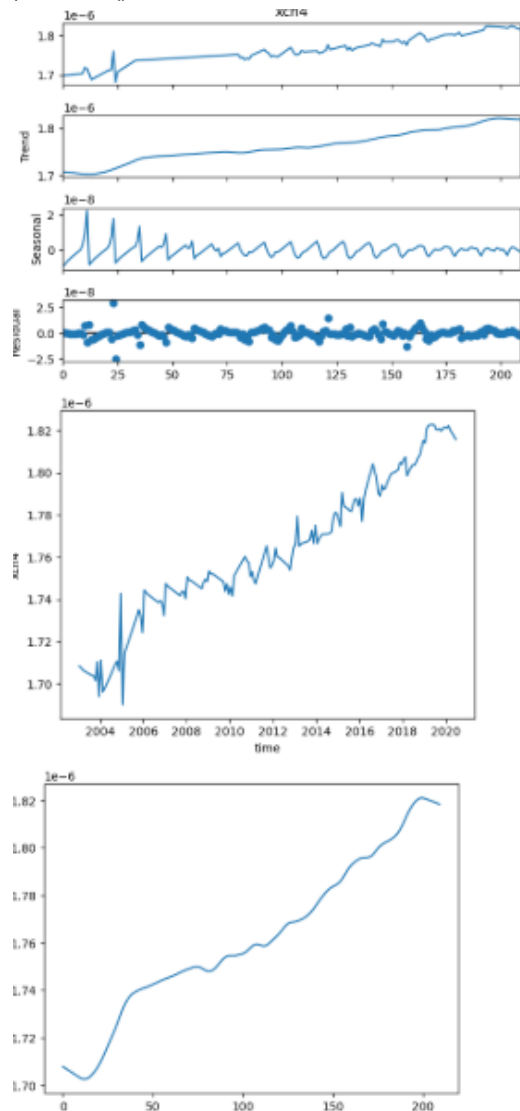
```
deseasonalized.plot()
```

```
plt.show()
```

直接用 trend

```
plt.plot(trend)
```

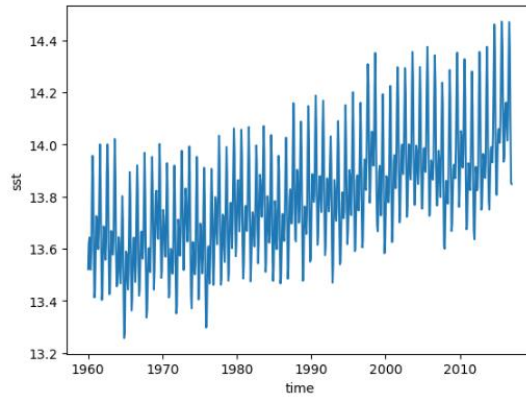
```
plt.show()
```



（最后使用分段线性插值结果）

2.

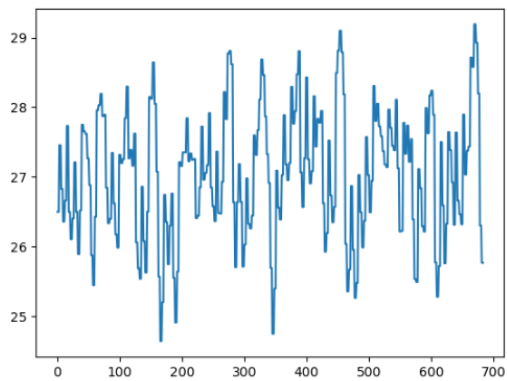
```
[145]: #plot global monthly SST time series
sst2 = sst.sst.mean(dim=['lat','lon'])
sst2.plot()
plt.show()
#总体呈缓慢而稳定的上升趋势，有很频繁的季节性剧烈波动，这个波动大多数情况下显得很整齐，在前半数据例如1960-1980平均值下半部分的部分月份数据比较杂乱，上升趋势内部有波动
```



全球

```
]： #展平为一维纯数值，去掉V/S分组标签，长度为684
SST3_flat = SST3.values.flatten()
SST3_flat

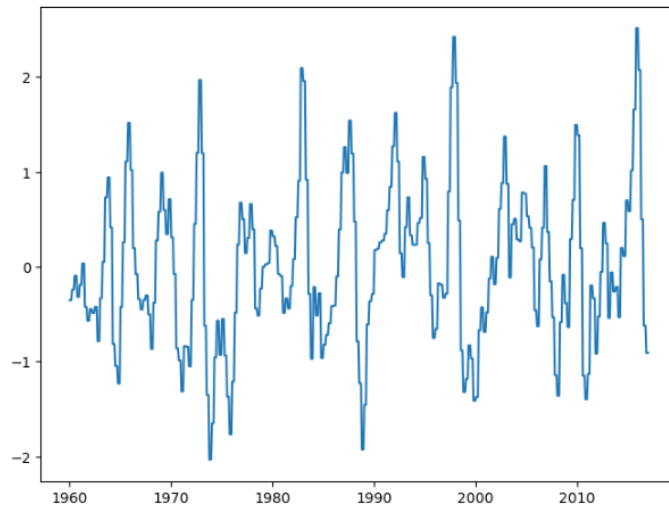
#得到滑动平均值图
plt.plot(SST3_flat)
plt.show()
```



滑动平均值

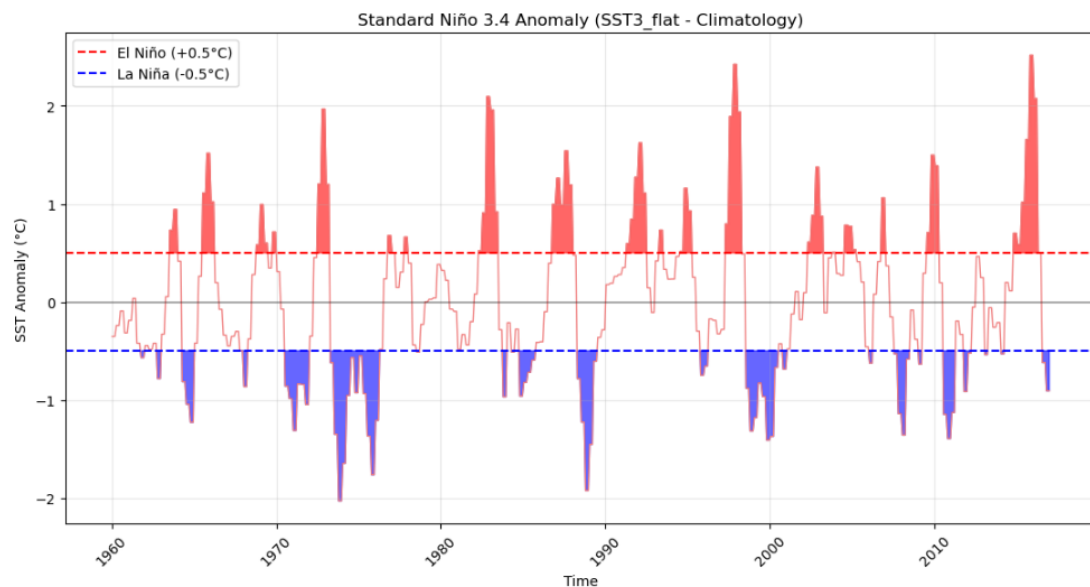
```
[151]: #不是很懂这个subtract climatology from SST time series to obtain anomalies是什么意思, 去问了AI
#气候态: 某一长期时期(如 1981-2010 年)内, 每个月份的平均海温(比如所有 1 月的平均、所有 2 月的平均, 而非整段时期的总平均)
# 给SST3_flat添加时间坐标(和SST的时间维度一致)
SST4 = xr.DataArray(
    SST3_flat,
    coords=({'time': SST.time}),
    dims=['time']
)
SST4
# 2. 按月份计算气候态(比如所有1月的滚动平均均值、所有2月的均值...)
climatology = SST4.groupby('time.month').mean(dim='time')
climatology

# 3. 计算标准异常值(滚动平均 - 气候态, 去掉季节循环)
SST4['month'] = SST.time.dt.month
sst_anom = SST4.groupby('time.month').map(lambda x: x - climatology.sel(month=x.month[0]))
sst_anom.plot(size=6)
plt.show()
# 比上面得到的平滑很多
```



异常值

2.2



画图结果

3.

数据

#3.1

reference: Huffman, G.J., A. Behrangi, D.T. Bolvin, E.J. Nelkin (2022), GPCP Version 3.2 Satellite-Gauge (SG) Combined Precipitation Data Set, Edited by Huffman, G.J., A. Behrangi, D.T. Bolvin, E.J. Nelkin, Greenbelt, Maryland, USA, Goddard Earth Sciences Data and Information Services Center (GES DISC), Accessed: [Data Access Date], 10.5067/MEASURES/GPCP/DATA304

尝试过用 wget 批量下载，报错
<https://blog.csdn.net/bulley/article/details/131313150#:~:text=%E6%96%87%E7%AB%A0%E4%B8%B8%E7%BB%8D%E4%BA%86%E5%A6%82%E4%BD%95%E5%88%A9%E7%94%A8wget%E5%B7%A5%E5%85%B7%E5%9C%A8Windows%E7%8E%AF%E5%A2%83%E4%B8%8B%E6%89%B9%E9%87%8F%E4%B8%8B%E8%BD%BDEarthdata%E7%9A%84%E6%95%B0%E6%8D%AE%EF%BC%8C%E5%8C%85%E6%8B%AC%E5%B0%86wget.exe%E5%92%8Cdownload.txt%E6%94%BE%E7%BD%AE%E5%9C%A8%E5%90%8C%E4%B8%80%E7%9B%AE%E5%BD%95%EF%BC%8C%E4%BD%BF%E7%94%A8cmd%E5%91%BD%E4%BB%A4%E8%A1%8C%E6%89%A7%E8%A1%8C%E4%B8%8B%E8%BD%BD%E6%93%8D%E4%BD%9C%EF%BC%8C%E4%BB%A5%E5%8F%8A%E4%BF%AE%E6%94%B9%E5%91%BD%E4%BB%A4%E4%B8%AD%E7%9A%84%E7%94%A8%E6%88%B7%E5%90%8D%E5%92%8C%E5%AF%86%E7%A0%81%E3%80%82,%E4%B8%8B%E8%BD%BD%E5%AE%8C%E6%88%90%E5%90%8E%EF%BC%8C%E8%BF%98%E6%8F%90%E4%BE%9B%E4%BA%86%E5%8E%BB%E9%99%A4nc%E6%95%B0%E6%8D%AE%E6%96%87%E4%BB%B6%E5%A4%8D%E6%9D%82%E5%90%8E%E7%BC%80%E7%9A%84%E6%96%B9%E6%B3%95%E3%80%82>

2022-2023 年 7 月的卫星降水数据

来源: <https://www.heywhale.com/mw/project/62660972af51920017414a26>
%cd C:\Users\YiChen\ps31

%ls *.nc4 #输出 nc 文件

source: 豆包-----

#读取过程（每次打开 python 只能运行一次，除非改 output 文件名）

1. 读取当前目录下所有 GPCP nc4 文件

print("正在查找当前目录下的 GPCP 文件...")

nc_files = [f for f in os.listdir(os.getcwd()) if f.startswith("GPCPMON_L3_") and f.endswith(".nc4")]

if not nc_files:

raise FileNotFoundError(

f"在当前目录 ({os.getcwd()}) 中未找到 GPCP 文件! \n"

"请确认: 1. 文件格式为 GPCPMON_L3_YYYYMM_V3.2.nc4; 2. 文件已放在

C:\Users\YiChen 目录下"

)

```
# 2. 修复文件名解析：用正则表达式提取 6 位年月数字（如 202101）
def extract_ym(filename):
    """ 用正则匹配文件名中的 6 位数字（YYYYMM 格式），适配
    GPCPMON_L3_202101_V3.2.nc4"""
    match = re.search(r'(\d{6})', filename) # 匹配 6 个连续数字
    if match:
        return match.group(1)
    else:
        raise ValueError(f"文件名 {filename} 中未找到 6 位年月数字（如 202201），请检查
        文件格式！")
```

```
# 按年月排序（修复后可正确排序）
nc_files.sort(key=lambda x: extract_ym(x))
nc_file_paths = [f for f in nc_files]
file_yms = [extract_ym(f) for f in nc_files]

# 显示文件信息（现在会正确显示时间范围）
print(f"✅ 找到 {len(nc_files)} 个文件")
print(f"现有文件时间范围：{file_yms[0]} → {file_yms[-1]}")
print(f"文件列表（按时间排序）：{nc_files}") # 可选：查看排序后的文件列表
```

```
# 3. 批量读取并合并文件
ds_list = []
for file_path in nc_file_paths:
    ds = xr.open_dataset(file_path, engine="netcdf4") # 小写引擎名
    ds_list.append(ds)
```

```
# 合并：覆盖元数据冲突，按 time 维度拼接
ds_combined = xr.concat(
    objs=ds_list,
    dim="time",
    combine_attrs="override" # 忽略元数据差异
)
```

```
# 4. 修正 time 坐标（确保与文件名一致）
times = pd.to_datetime(file_yms, format="%Y%m")
ds_combined = ds_combined.assign_coords(time=times)
```

```
# 5. 变量名校验
var_name = ["gauge_precip", "satellite_precip"]
missing_vars = [var for var in var_name if var not in ds_combined.variables]
if missing_vars:
    print(f"\n❌ 以下变量不存在于文件中：{missing_vars}")
```



```

print(f'文件中可用变量: {list(ds_combined.variables.keys())}')
# 让用户补充正确的变量名 (如果有拼写错误)
for var in missing_vars:
    new_var = input(f'请输入 '{var}' 对应的正确变量名 (复制上面列表中的一个): ')
    var_names[var_name.index(var)] = new_var # 替换为正确变量名
# 只保留两个目标变量
ds_combined = ds_combined[var_name]

```

```

1: # 6. 保存文件
output_nc = "GPCP_gauge_precip_timeseries_202101-202307.nc4"

# 保存两个变量 (分别配置压缩)
encoding = {var: {"zlib": True, "complevel": 1} for var in var_name}
ds_combined.to_netcdf(
    output_nc,
    engine="netcdf4",
    encoding=encoding
)

print(f" 输出文件路径: {output_nc}")
print(f" 数据维度: {ds_combined.sizes}")

200     try:
201         yield file

File D:\anaconda3\lib\site-packages\xarray\backends\file_manager.py:217, in CachingFileManager._acquire_with_cache_info(self, needs_lock)
215     kwargs = kwargs.copy()
216     kwargs["mode"] = self._mode
--> 217 file = self._opener(*self._args, **kwargs)
218 if self._mode == "w":
219     # ensure file doesn't get overridden when opened again
220     self._mode = "a"

File src\netCDF4\_netCDF4.pyx:2517, in netCDF4\_netCDF4.Dataset.__init__()
File src\netCDF4\_netCDF4.pyx:2154, in netCDF4\_netCDF4._ensure_nc_success()

PermissionError: [Errno 13] Permission denied: 'C:\Users\ViChen\ps31\GPCP_gauge_precip_timeseries_202101-202307.nc4'
Fix Code

```

可能出现 premission denied (我把存下来的文件放在目录下的 ps31 文件夹了, 可能读取目录会有混乱, 同时每次打开 python 只能读一次, 不然会因为旧文件导致 premission denied)

```

正在查找当前目录下的GPCP文件...
✅ 找到 31 个文件
现有文件时间范围: 202101 → 202307
文件列表 (按时间排序): ['GPCPHON_L3_202101_V3.2.nc4', 'GPCPHON_L3_202102_V3.2.nc4', 'GPCPHON_L3_202103_V3.2.nc4', 'GPCPHON_L3_202104_V3.2.nc4', 'GPCPHON_L3_202105_V3.2.nc4', 'GPCPHON_L3_202106_V3.2.nc4', 'GPCPHON_L3_202107_V3.2.nc4', 'GPCPHON_L3_202108_V3.2.nc4', 'GPCPHON_L3_202109_V3.2.nc4', 'GPCPHON_L3_202110_V3.2.nc4', 'GPCPHON_L3_202111_V3.2.nc4', 'GPCPHON_L3_202112_V3.2.nc4', 'GPCPHON_L3_202201_V3.2.nc4', 'GPCPHON_L3_202202_V3.2.nc4', 'GPCPHON_L3_202203_V3.2.nc4', 'GPCPHON_L3_202204_V3.2.nc4', 'GPCPHON_L3_202205_V3.2.nc4', 'GPCPHON_L3_202206_V3.2.nc4', 'GPCPHON_L3_202207_V3.2.nc4', 'GPCPHON_L3_202208_V3.2.nc4', 'GPCPHON_L3_202209_V3.2.nc4', 'GPCPHON_L3_202210_V3.2.nc4', 'GPCPHON_L3_202211_V3.2.nc4', 'GPCPHON_L3_202212_V3.2.nc4', 'GPCPHON_L3_202301_V3.2.nc4', 'GPCPHON_L3_202302_V3.2.nc4', 'GPCPHON_L3_202303_V3.2.nc4', 'GPCPHON_L3_202304_V3.2.nc4', 'GPCPHON_L3_202305_V3.2.nc4', 'GPCPHON_L3_202306_V3.2.nc4', 'GPCPHON_L3_202307_V3.2.nc4']

# 6. 保存文件
output_nc = "GPCP_gauge_precip_timeseries_202101-2023071.nc4"

# 保存两个变量 (分别配置压缩)
encoding = {var: {"zlib": True, "complevel": 1} for var in var_name}
ds_combined.to_netcdf(
    output_nc,
    engine="netcdf4",
    encoding=encoding
)

print(f" 输出文件路径: {output_nc}")
print(f" 数据维度: {ds_combined.sizes}")

输出文件路径: GPCP_gauge_precip_timeseries_202101-2023071.nc4
数据维度: Frozen({'time': 31, 'lat': 360, 'lon': 720})

```

```
[161]: # Load the netCDF4 file
prep = xr.open_dataset("GPCP_gauge_precip_timeseries_202101-202307.nc4", engine="netcdf4")
prep
```

- Dimensions: (time: 31, lat: 360, lon: 720)
▼ Coordinates:

lon	(lon)	float32	-179.8 -179.2 ... 179.2 179.8	
lat	(lat)	float32	89.75 89.25 88.75 ... -89.25 -89.75	
time	(time)	datetime64[ns]	2021-01-01 ... 2023-07-01	

▼ Data variables:

satellite_precip	(time, lat, lon)	float32	...	
------------------	------------------	---------	-----	--

- Indexes: (3)
- Attributes: (66)

```
[162]: # plot the monthly time series
prep1=prep.gauge_precip.mean(dim=['lon','lat'])
prep1.plot()
plt.show()
```

```
[176]: # Load the netCDF4 file
prep = xr.open_dataset("GPCP_gauge_precip_timeseries_202101-2023071.nc4", engine="netcdf4")
prep
```

- Dimensions: (time: 31, lat: 360, lon: 720)
▼ Coordinates:

lon	(lon)	float32	-179.8 -179.2 ... 179.2 179.8	
lat	(lat)	float32	89.75 89.25 88.75 ... -89.25 -89.75	
time	(time)	datetime64[ns]	2021-01-01 ... 2023-07-01	

▼ Data variables:

gauge_precip	(time, lat, lon)	float32	...	
satellite_precip	(time, lat, lon)	float32	...	

- Indexes: (3)

正常运行结果

#用 stl 去季节性

```
stl1 = STL(prepare, seasonal = 13, period = 12)
result2 = stl1.fit()
result2.plot()
plt.show()
```

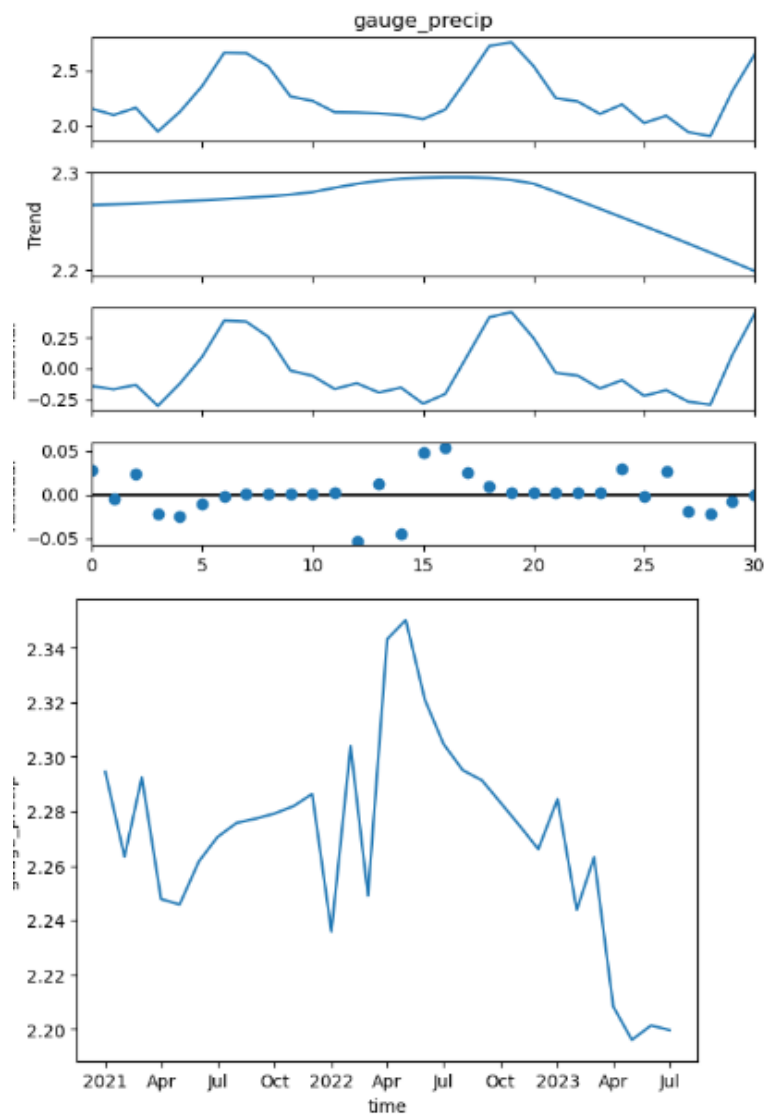
分解结果

```
trend1 = result2.trend
seasonal1 = result2.seasonal
residual1 = result2.resid
```

4. 去除季节性（原始数据 - 季节性成分）

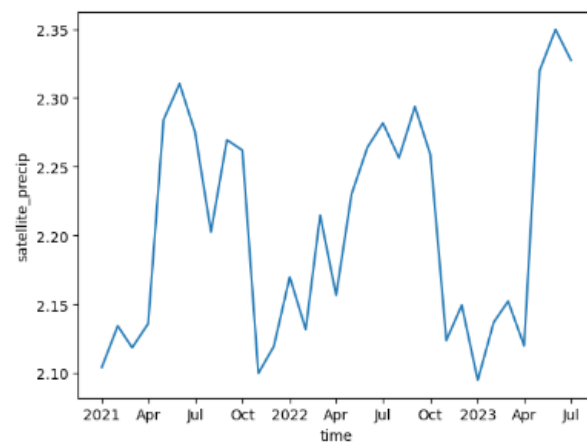
```
deseasonalized1 = prepare - seasonal1
deseasonalized1.plot()
plt.show()
```

#去季节性后，年平均降水在 2.27mm/day 左右上下波动，2023 年 4-6 月降水较多，2023 年 4-6 月降水较以往两年较少



#3.2

```
#1.# plot the monthly satellite time series
prep2=prep.satellite_precip.mean(dim=['lon','lat'])
prep2.plot()
plt.show()
#趋势一停，4-10月降水甚多（为什么）
#数值不同，有甚多波动，平均温度较gauge低（推测原因是gauge是风浪调整后测得的降水量）
```

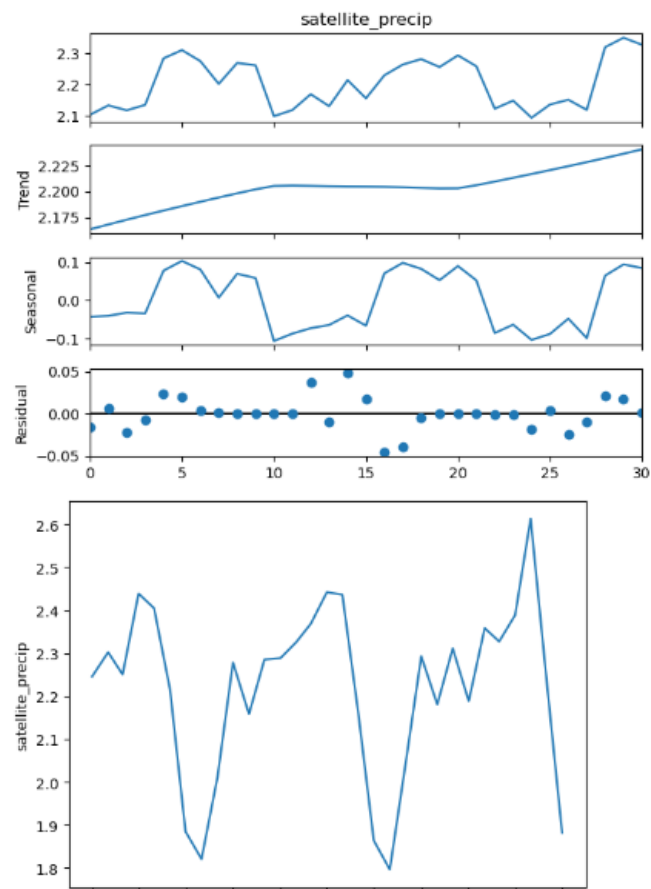


```
#2.
#用stl去季节性
stl2 = STL(prep2, seasonal = 13, period = 12)
result2 = stl2.fit()
result2.plot()
plt.show()
```

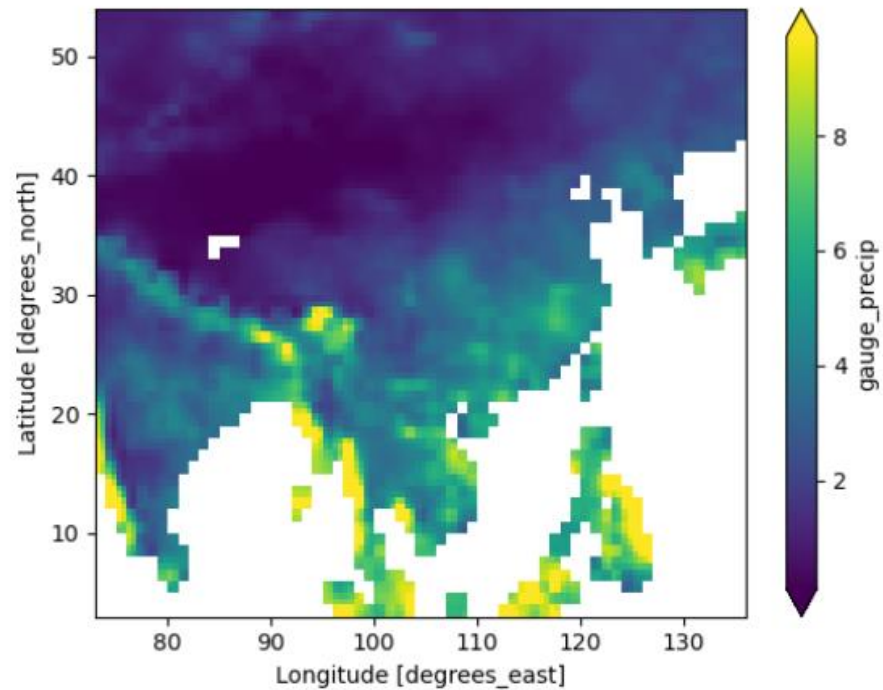
```
# 分解结果
trend2 = result2.trend
seasonal2 = result2.seasonal
residual2 = result2.resid
```

```
# 4. 去除季节性 (原始数据 - 季节性成分)
deseasonalized2 = prep2 - seasonal1
deseasonalized2.plot()
plt.show()
```

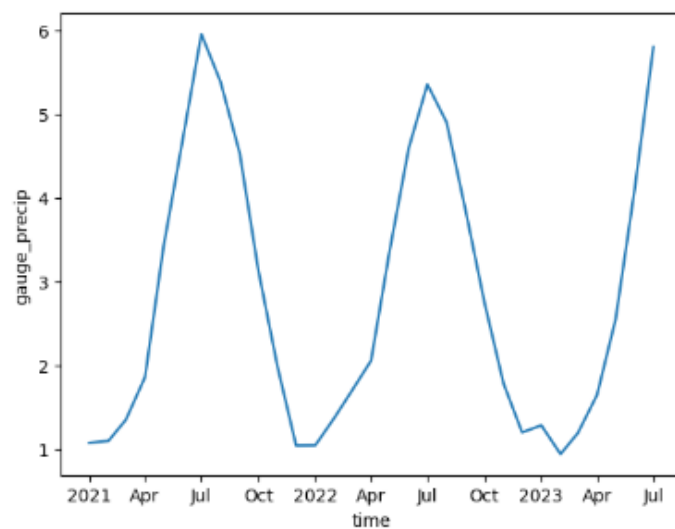
去季节性后，年平均降为什么还是有季节性波动？感觉trend更能显示去季节性后变化趋势



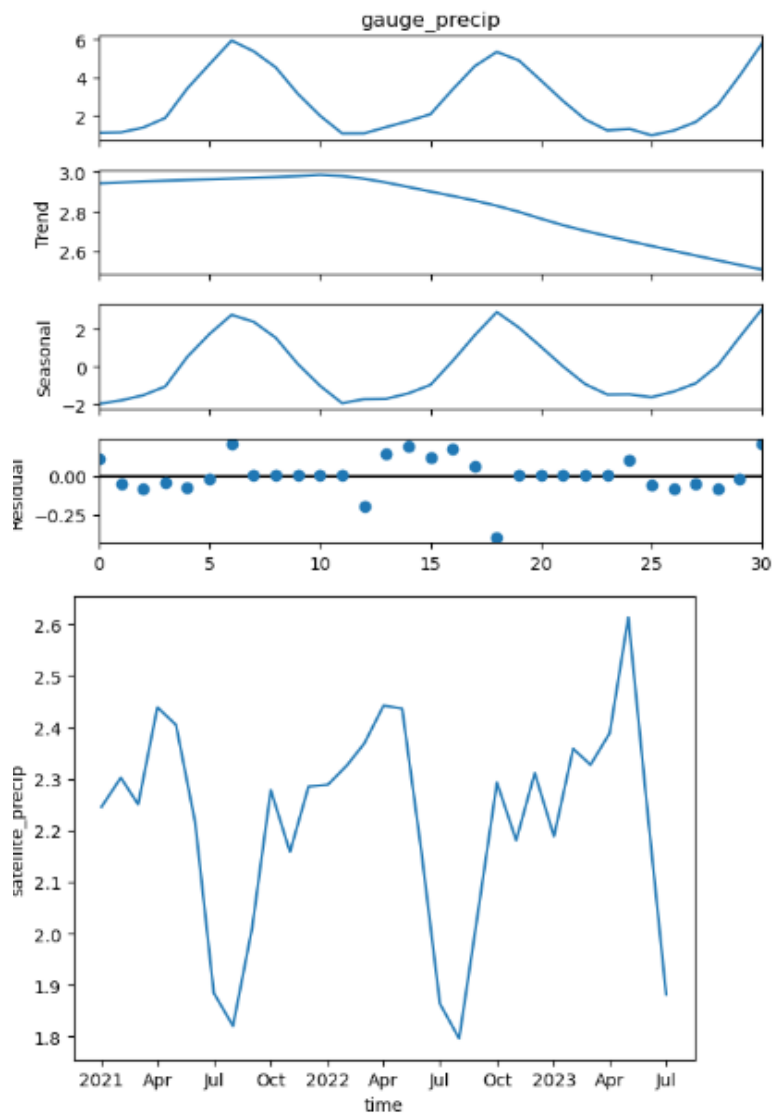
```
#3 mean precipitation of China
gauge_precip1 = prep.gauge_precip.sel(lon=slice(73,136),lat=slice(54,3))
gauge_precip1.mean(dim='time').plot(robust=True)
plt.show()
gauge_precip1
```



```
#4 time series precipitation of China
china = gauge_precip1.mean(dim=['lat','lon'])
china.plot()
plt.show()
#最高可达到6，最低1mm/day，峰值高，季节性特征明显，波幅更剧烈
```



Stl 去季节性后:



```
xarray.DataArray 'satellite_precip'
```

```
array(2.2194921)
```

```
- Coordinates: (0)
```

```
- Indexes: (0)
```

```
- Attributes: (0)
```

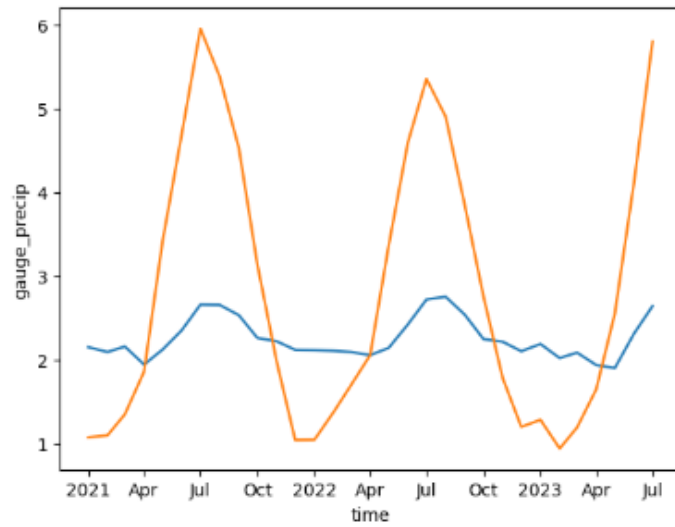
```
#compare global and Chinese gauge data
```

```
prepl.plot()
```

```
china.plot()
```

```
plt.show()
```

```
#趋势一样（为什么是北半球为全球趋势？），中国波动比全球剧烈得多（可能中国纬度较低，同时南半球数据的中和有极大影响）
```



5.